

Software Architecture Assignment 4: Architectural Decision-Making and Justification

Group Members: Kashkenov Madiyar, Toktarov Nurislam

1. Introduction and Case Summary

Selected System

The selected system is a Real-Time Geolocation-Based Service Worker Marketplace Platform, continued consistently from Assignment 1 and Assignment 2.

System Purpose and Main Users

The platform connects customers who require immediate or scheduled services (e.g., plumbing, electrical work, cleaning, maintenance) with nearby service workers using real-time geolocation. Customers can search for nearby workers, post service requests, track job progress, communicate in real time, and complete secure payments. Service workers manage profiles, receive job notifications, and track earnings. Administrators oversee system health, analytics, dispute resolution, and compliance.

The system is designed to operate at scale, supporting thousands of concurrent users while ensuring high performance, reliability, security, and maintainability.

2. Decision Drivers

2.1 Business Drivers

The primary business drivers are rapid time-to-market, cost efficiency, and scalability for growth. The platform must reach market quickly to remain competitive, while keeping infrastructure costs predictable. Expected user growth is significant, requiring an architecture that can scale horizontally without major redesign. High availability is critical, as downtime directly impacts revenue and user trust.

2.2 Technical Drivers

Key technical drivers are derived from prioritized quality attributes defined in Assignment 2. Performance efficiency and reliability are critical due to real-time geolocation and booking workflows. Security is mandatory because the system processes personal data and payments. Maintainability is essential to support frequent feature updates and independent service evolution within a microservices environment.

2.3 Organizational Drivers

The development team is small-to-medium sized with strong backend skills and growing DevOps maturity. The organization favors technologies with strong ecosystem support, clear documentation, and automation-friendly tooling. Operational capabilities include CI/CD pipelines, containerization, and cloud-native monitoring.

2.4 Constraints

Hard constraints:

- Compliance with GDPR and PCI-DSS
- Encrypted communication and secure authentication
- Microservices architecture assumption from previous assignments

Preferences (flexible):

- Open-source technologies
- Cloud-first deployment
- Language and frameworks with strong performance characteristics

3. Architectural Decision Records (ADRs)

ADR Template Used: Michael Nygard's ADR template

ADR 1: Architectural Style Selection

Context

The system must support independent scaling, high availability, and rapid feature evolution. Performance and reliability are critical quality attributes. The architecture must handle real-time communication and variable traffic loads.

Decision

Adopt a Microservices Architecture with an API Gateway.

Alternatives Considered

- **Monolithic Architecture:** Simple to develop initially but limits scalability and independent deployment.
- **SOA:** Provides separation but introduces heavier governance and complexity.
- **Event-Driven Architecture:** Powerful for decoupling but adds operational complexity.

Consequences

- **Positive:** Independent scaling, fault isolation, faster deployments.
- **Negative:** Increased operational complexity and distributed system overhead.

Rationale

Microservices best support scalability, maintainability, and availability requirements while aligning with organizational DevOps capabilities.

ADR 2: Programming Language Selection

Context

High concurrency, low latency, and efficient resource usage are required to support real-time geolocation updates and WebSocket connections.

Decision

Use Go (Golang) as the primary backend language.

Alternatives Considered

- **Java:** Mature ecosystem but higher memory overhead.
- **Node.js:** Fast development but weaker CPU-bound performance.

Consequences

- **Positive:** High performance, efficient concurrency, low memory footprint.
- **Negative:** Smaller ecosystem compared to Java.

Rationale

Go provides strong performance and concurrency support, directly addressing performance efficiency and scalability drivers.

ADR 3: Framework Selection

Context

The system requires lightweight, high-performance HTTP handling and clear API definitions.

Decision

Use the Gin web framework for backend services.

Alternatives Considered

- **Echo:** Similar performance, smaller community.
- **Fiber:** Very fast but less mature ecosystem.

Consequences

- **Positive:** High throughput, middleware support, simplicity.
- **Negative:** Fewer built-in features than heavier frameworks.

Rationale

Gin balances performance and developer productivity while remaining lightweight.

ADR 4: Database Selection

Context

The system relies heavily on complex geospatial queries and strong data consistency for compliance and auditing.

Decision

Use PostgreSQL with PostGIS, supplemented by Redis for caching.

Alternatives Considered

- **MongoDB:** Flexible schema but weaker spatial querying.
- **Pure NoSQL:** High scalability but weaker consistency guarantees.

Consequences

- **Positive:** Accurate geospatial queries, ACID compliance.
- **Negative:** Requires scaling strategies for high write loads.

Rationale

PostgreSQL with PostGIS best satisfies accuracy, consistency, and compliance requirements.

ADR 5: Infrastructure and Deployment Strategy

Context

High availability, scalability, and automated deployments are required.

Decision

Deploy on Cloud Infrastructure (IaaS/PaaS) using Docker containers and Kubernetes.

Alternatives Considered

- **On-premise:** High control but limited scalability.
- **Serverless:** Reduced ops but limited control over long-lived connections.

Consequences

- **Positive:** Elastic scaling, self-healing, automation.
- **Negative:** Increased operational complexity.

Rationale

Container orchestration supports availability, scalability, and DevOps maturity goals.

ADR 6: Integration and Communication Strategy

Context

The system requires both real-time communication and reliable service-to-service interaction.

Decision

Adopt a **Hybrid Communication Strategy**:

- REST for synchronous requests
- WebSockets and messaging (Redis Pub/Sub or Kafka) for asynchronous events

Alternatives Considered

- **Pure synchronous REST:** Simpler but less scalable.
- **Pure asynchronous messaging:** Complex for request-response flows.

Consequences

- **Positive:** Low latency, scalability, flexibility.
- **Negative:** Increased system complexity.

Rationale

Hybrid communication best balances performance and reliability needs.

4. Trade-offs Summary

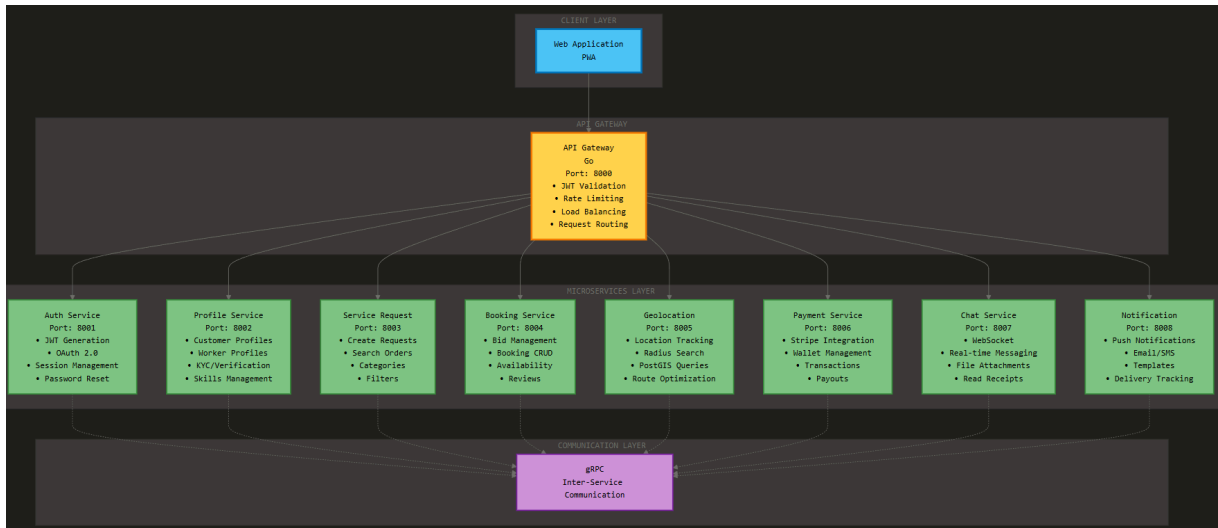
- 1. **Performance vs. Maintainability**
Caching and asynchronous messaging improve performance but increase system complexity.
- 2. **Availability vs. Cost Efficiency**
Redundancy and multi-zone deployment increase costs but ensure uptime.
- 3. **Security vs. Usability**
Strong authentication adds friction but is necessary for compliance.

5. Risks and Mitigation Strategies

Risk	Likelihood	Impact	Mitigation
Performance degradation under peak load	Medium	High	Load testing, caching, autoscaling
Operational complexity of microservices	Medium	Medium	Automation, documentation
Security vulnerabilities	Low	High	Audits, monitoring
Team skill gaps	Medium	Medium	Training, code reviews
Integration failures	Medium	Medium	Circuit breakers, retries

6. Architectural Overview

The system is composed of independently deployable microservices accessed through an API Gateway. Core services include User Management, Geolocation, Booking, Payments, and Notifications. PostgreSQL with PostGIS stores transactional and spatial data, while Redis supports caching and real-time messaging. Services communicate via REST and asynchronous channels, and the entire system is deployed on a cloud-based container platform.



7. Reflection and Evolution

The architecture effectively addresses prioritized quality attributes, particularly performance, reliability, and security. The microservices approach allows scaling from MVP to full production while maintaining flexibility for future changes. However, it introduces operational complexity that must be managed through automation and observability. Overall, the architecture provides a strong foundation for long-term system evolution.

8. Assumptions and Open Questions

Assumptions:

- Cloud infrastructure remains available and cost-effective
- Traffic growth follows projected patterns

Open Questions:

- Should premium users receive higher QoS guarantees?
- How will future regulatory changes affect data retention policies?